

Tracing & Logging

In the previous architecture, AstroLearn relied on basic telemetry with individual health checks and local logs, but it lacked centralised logging and had no way to trace a request across multiple services. Each microservice stored its information independently, making it difficult to understand the behaviour of the system as a whole and therefore harder to diagnose issues.

In the updated design (Fig. 1), the observability layer has been expanded to include centralised logging and distributed tracing. Every request entering the system through the API Gateway is assigned a correlation ID, a unique identifier that is passed to all subsequent internal calls and to events published through the Event Bus. All components inside the shaded area shown in Figure 1 send now their logs, traces and health data to the Observability component. This provides a complete view of how each request flows across the system. Logging is applied at key points, such as request entry and exit, state updates, event publication or consumption, and error handling to avoid unnecessary noise while preserving essential diagnostic information. For example, when a student completes a quiz, the Quiz service logs the event with the correlation ID, the Progress service records the update triggered by the event, and the Notification service logs the message delivery.

Finally, these improvements follow the monitoring principles described by Newman (2015), providing significantly better visibility of system behaviour, allowing logs from multiple services to be linked through the same correlation ID and making it easier to identify bottlenecks and detect failures.

Scaling

- Horizontal Scaling & Load Balancing

Previously, the system relied on single instances of the API Gateway and each microservice, which made it more vulnerable to high traffic and individual failures. The new architecture (see figure 1) follows the horizontal-scaling techniques, which Newman (2015) notes are commonly adopted by large-scale platforms, such as Netflix and Google, adapted here for a lightweight educational application. A load balancer is now placed in front of multiple API Gateway instances, allowing incoming requests to be distributed across them and removing the gateway as a likely point of failure. Additionally, the most frequently used user-facing services: Auth, Quiz, User Profile and Progress, are also deployed in multiple instances, so that traffic can be routed to any available copy. In addition, Service Discovery operates as a highly available cluster, as recommended by Newman (2015), preventing it from becoming a bottleneck and ensuring that every request is routed only to healthy microservice instances

when needed. Together, these changes create a more resilient and distributed execution path than the previous design.

Moreover, the **External Data Importer** does not require horizontal scaling, but it can be scaled in the same way if future requirements demand higher ingestion throughput.

- **Autoscaling & Resilience**

Autoscaling is also introduced to ensure that AstroLearn can handle fluctuations in demand without manual intervention. Each replicated service can automatically scale out when resource usage increases and scale in when traffic decreases, allowing the system to remain responsive while avoiding unnecessary resource consumption. Autoscaling additionally improves resilience, as new instances can be created if existing ones become unavailable. This mechanism is particularly useful during peak periods, such as when many students complete quizzes simultaneously.

By dynamically adjusting the number of instances, the system maintains stable performance and availability without requiring changes to the application logic. According to Newman (2015), this approach enhances elasticity and fault tolerance.

- **Data Scaling & Consistency**

The updated design also improves scalability at the data layer. Instead of replicating read-heavy services such as Catalog, News and Sky Events, they continue to run as single service instances, but their underlying database now uses read replicas. Moreover, Newman (2015) states that read replicas allow the system to distribute read requests across several database copies, reducing load on the primary database and improving performance during periods of high activity. This model relies on eventual consistency (GeeksforGeeks, 2025), where replicas may briefly return older data before receiving the latest update.

For example, if a student completes a quiz and the Progress service updates the score in the primary database, a replica may momentarily show the previous value before converging to the correct state.

Together, these improvements enhance data availability and read throughput without increasing complexity. Moreover, GeeksforGeeks (2025) states that this approach is lightweight, avoids modifying service logic and therefore is appropriate for applications like AstroLearn, where minor delays in data propagation are acceptable and do not affect user experience. In conclusion, the current shared database with logical separation supports this approach, and the system can evolve toward a full database-per-service structure if the application grows.

Advanced Security

In the previous architecture, AstroLearn relied on a basic security model centred on the Auth Service and simple protected routes, lacking the strategies needed to secure a growing microservice system. Since the application may be adopted in institutional educational contexts, such as schools and universities, the updated design incorporates advanced security strategies.

The new architecture introduces an external identity provider, a single sign-on gateway flow and a token-based zero-trust model with fine-grained authorization that secures both synchronous and asynchronous communication across the platform, while improving the protection of sensitive configuration data. More specifically, the authentication flow is fortified by introducing an external Identity Provider, as shown in Figure 1. This dedicated service verifies user credentials and issues identity tokens. For example, when Anna, a student, attempts to access the system, the request reaches the API gateway, which redirects her to the Identity provider for login. After verifying her credentials, the Identity Provider returns an identity token, such as *Anna – UserID123*, to the gateway (Patel, 2023).

Once this token is received, the Gateway contacts the Auth Service to retrieve Anna's application role, such as student or teacher. Using this information, the Gateway issues a signed internal token that will be attached to all subsequent requests and events inside the system. This token carries key attributes, such as user ID and role, and serves as a trusted representation of the user for the rest of the platform. This approach follows the Single Sign-On Gateway pattern, and the token propagation and service-to-service authentication principles explained by Newman (2015), where services validate the credentials attached to a request instead of trusting internal traffic by default. By centralising authentication and token issuance at the Gateway, the system establishes a secure and consistent foundation for internal communication. When Gateway issues the signed internal token, all communication inside the system relies on it. A limitation of the previous design was that internal requests were implicitly trusted, meaning that a compromised service could access others without restriction. The updated architecture adopts a zero-trust model, where no microservice is automatically trusted and every internal request must carry the signed token created by the Gateway. Each service validates the token before processing a call, ensuring that only authorised operations can be performed. For example, when Anna interacts with the Progress Service, the service reads her role from the token and applies fine-grained authorization rules. Therefore, as a student, Anna can view only her own results, while future roles, such as teachers could be allowed to view additional data.

Finally, the same principle applies to asynchronous communication. When the Quiz Service publishes a *QuizCompleted* event, the event includes the same signed token, that indicates the identity and the permissions of the role, allowing the Event Bus to deliver it only to authorised subscribers such as Progress or Notifications. This integrated zero-trust approach across all internal interactions strengthens the security baseline and prevents unauthorised access even if a service is compromised.

Bus Architecture

Previously, several microservices communicated through internal REST calls. This approach creates tight coupling between services and offers no buffering if a target service is unavailable. As interactions increase, this communication model becomes increasingly difficult to scale and maintain, therefore it needs the implementation of additional strategies.

In the updated architecture, an event bus has been introduced, see Figure 1, to support asynchronous publish/subscribe communication. This component sits between the microservices, receiving events from publishers and delivering them to the appropriate subscriber services. This removes direct dependencies, as services no longer need to know or call each other. For example, the Quiz service now publishes a *'QuizCompleted'* event to the event bus, which then forwards it to Progress and Notification services for processing. Similarly, News and Sky Events publish events when new content becomes available, and the Notification service is subscribed to receive these updates.

If the subscriber is temporarily unavailable, the message remains in the event bus queue and is delivered once the service recovers. This event-based communication makes the backend more reliable, decoupled and easier to extend, allowing new services to subscribe to existing events without modifying the current microservices.

To conclude, a message broker such as RabbitMQ is appropriate for implementing the event bus in AstroLearn, as Iduwe (2022) notes this is a lightweight, open-source and simple to operate solution, making it suitable for smaller applications. Larger organisations may adopt more advanced platforms such as Apache Kafka or commercial integration tools like Mulesoft, which follow the same event-driven principles at a greater scale.

References:

GeeksforGeeks. (2025). *Eventual Consistency in Distributed Systems | Learn System Design*.

<https://www.geeksforgeeks.org/system-design/eventual-consistency-in-distributive-systems-learn-system-design/>

Iduwe, B. (2022). *Building an event bus with RabbitMQ*. Medium.

<https://bencoderus.medium.com/building-an-event-bus-with-rabbitmq-fa8ac5e7d19c>

Newman S. (2015). *Building microservices: designing fine-grained systems*. O'Reilly Media, Incorporated.

<https://ebookcentral.proquest.com/lib/think/reader.action?c=UERG&docID=1938300&ppg=1>

Patel, A. (2023). *Identity Providers — Types of Tokens in OpenID Connect and OAuth2*

Contexts. Medium. <https://medium.com/dotnet-hub/identity-providers-types-of-tokens-in-openid-connect-and-oauth2-overview-idtoken-accesstoken-refresh-token-68972ce3b1aa>